

Penetration Testing

Talisman (Wallet)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test scope	7
Security Rating	8
Code Quality	9
Penetration Test Resources	9
Dependencies	9
Severity Definitions	10
Penetration Test Findings	11
Centralisation	14
Conclusion	15
Our Methodology	16
Disclaimers	18
About Hashlock	19

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Talisman team partnered with Hashlock to conduct a security Penetration Test of their Talisman Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Talisman is a crypto wallet and asset management dashboard that unlocks a new world of web3 applications for Ethereum and Polkadot.

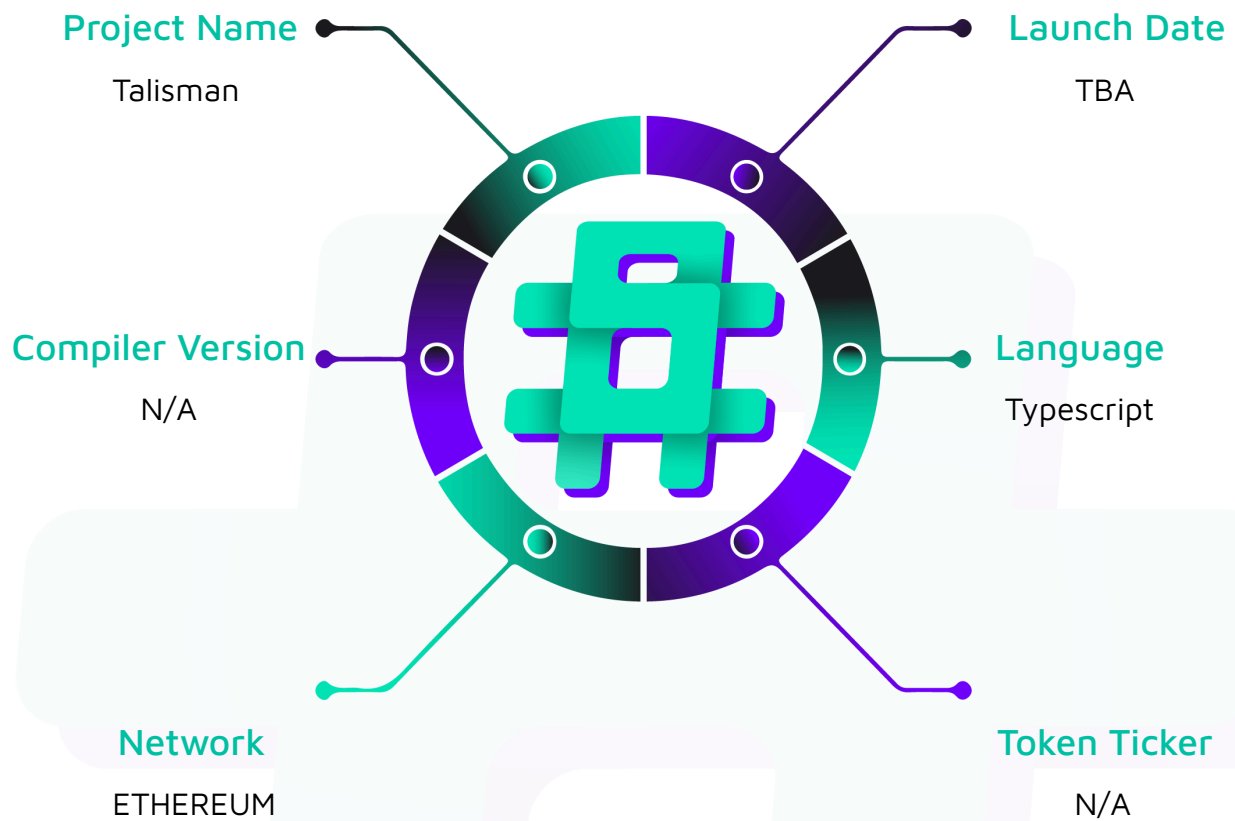
The Talisman browser extension allows you to safely store, send, receive, and harness your assets across relaychains, parachains and everything EVM. The Talisman web app allows you to seamlessly manage your assets in greater detail. Keep track of all your NFTs across all chains and explore new dApps and interesting projects.

Project Name: Talisman

Website: <https://app.talisman.xyz/>

Logo:



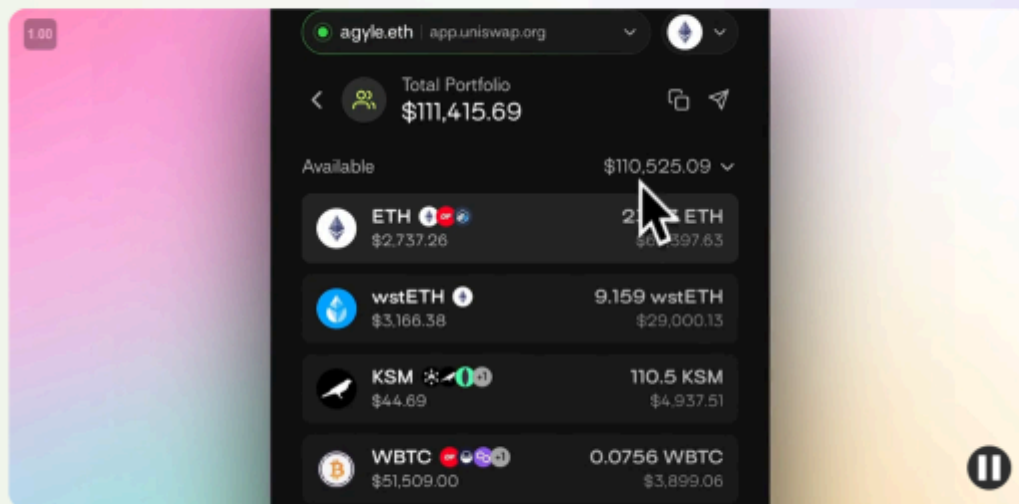
Visualised Context:

Project Visuals:

Multi-Chain Made Easy

Talisman is an Ethereum and Polkadot wallet that makes web3 simple for beginners and unlocks superpowers for pros.

Download Wallet



Try the Talisman wallet

Available on all good browsers



Add to Browser

Works with External Devices

Talisman works seamlessly with your favourite external devices like Ledger and Polkadot Vault.



Stake ETH, DOT, and More

With just a few clicks you can easily earn passive income on supported assets like: DOT, ETH, TAO, ASTR, and more.



#hashlock.

Hashlock Pty Ltd

Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the Talisman project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Talisman Project code base
Platform	N/A
Penetration Test Date	December, 2024
Repository	https://github.com/TalismanSociety/talisman/tree/dev/apps
Repository 2	https://github.com/TalismanSociety/talisman/tree/dev/packages
GitHub Commit Hash	53c7590a337cb71f2acc57a32f62c058912e1ec5
GitHub Commit Hash 2	b5a3f7ddc7f3a342611e67a9b62db6405bffd6e8

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

1 QAs

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This Penetration Test scope involves the code bases of the Talisman project, as outlined in the Penetration Test Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Penetration Test Resources

We were given the Talisman projects code base code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this code base's infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external code base calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

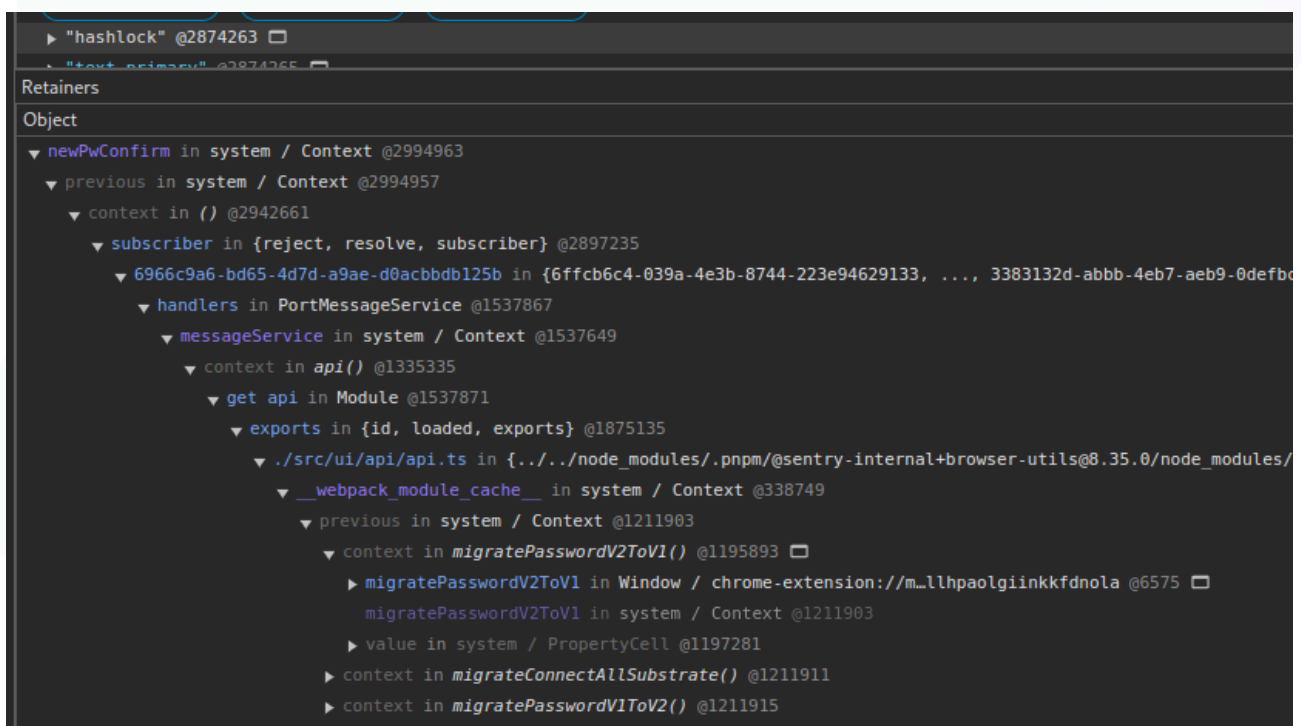
Penetration Test Findings

Medium

[M-01] ChangePasswordPage - Password stored in memory after password change

Description

During a password change operation, the front end UI subscribes to the API messages to receive the result of the password change operation. After the operation is done the new password value lingers in the extension memory until the page extension page is closed even though it is no longer needed.



Impact

A malicious actor with access to a user's computer can extract the plain text password from the browser memory.

Recommendation

After the password change dialog closes the variables related to it should be cleared from the memory to limit the effects of a possible breach of user information.

Status

Resolved

Low**[L-01] Authorised Sites - Talisman web app trusted by default****Description**

Hashlock has determined that the talisman web app hosted at app.talisman.xyz, used for operations such as staking, is trusted by the wallet extension implicitly. New accounts created by the extensions are also added to the list of addresses the website is allowed to see and create transaction requests with. During the onboarding process, the user is not notified of this. This behaviour is not present in similar wallet projects.

Impact

The wallet trusting the Talisman domain by default creates new attack vectors for hackers. Malicious actors that possibly breach the web app can use this trust to mislead users into signing unwanted transactions. Users are more likely to be misled since the relationship between the web app and the wallet is not made clear during the onboarding.

Recommendation

Information regarding the Talisman web app (the fact that it is separate from the wallet itself, its capabilities, etc) should be communicated to the users during the onboarding.

Status

Acknowledged

QA

[Q-01] ParaverseProtector - Hardcoded urls

Description

When referring to the Talisman website at app.talisman.xyz in the code, all instances but one use the constant defined in constants.ts:

```
export const TALISMAN_WEB_APP_DOMAIN = "app.talisman.xyz"
```

However, the allow list in paraverseprotector.ts uses a hardcoded value:

```
const DEFAULT_ALLOW = ["talisman.xyz", "app.talisman.xyz"]
```

Recommendation

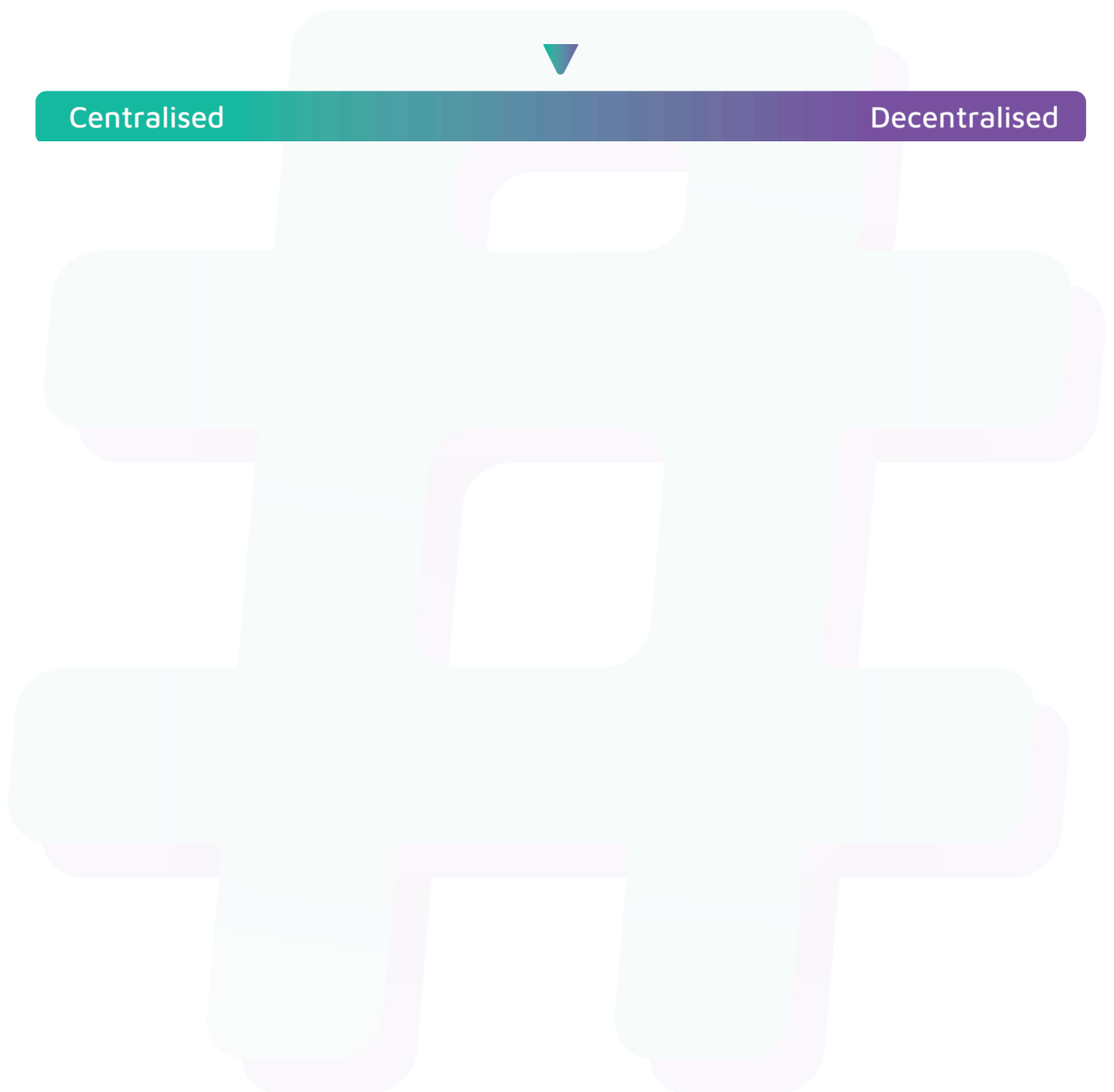
Switching to the web app domain constant can prevent unintended problems from arising if the domain is changed in the future.

Status

Resolved

Centralisation

The Talisman project values security and utility over decentralisation.



Conclusion

After Hashlock's analysis, the Talisman project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.